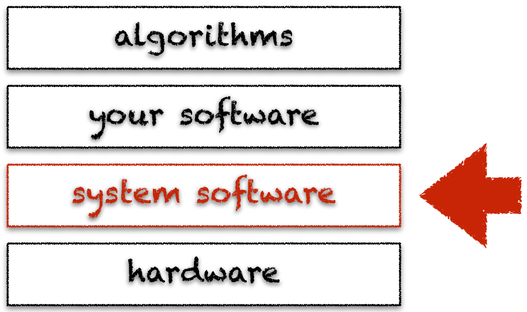
system software



learning objectives



- understand the role of an operating system
- understand the role of interpreters and compilers
- understand the role of runtime systems & libraries

what's system software?

application software consists in programs that
help to solve a particular computing problem,
e.g., write documents, browse the web, etc.

system software consists in programs that sit
between application software and the hardware,
providing common services to application software

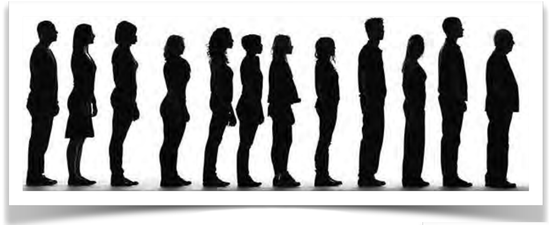
example of system software

- operating systems, game engines
- compilers, interpreters, virtual machines
- language runtimes, standard libraries

bits of history

1940s
1950s ◆ no system software

1960s ◆ batch systems



the
waiting era

1970s ◆ multi-user & time-sharing



the
sharing era

1980s ◆ personal desktop computers



the
personal era

1990s ◆ distributed systems



the
communication era

2000s ◆ mobile systems

2010s ◆ ubiquitous systems

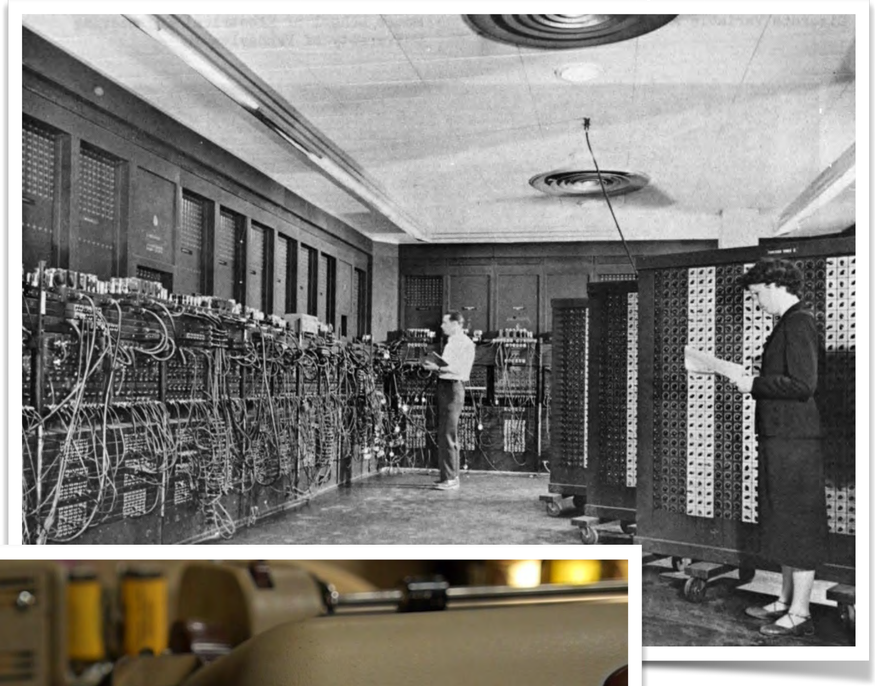


the digital
transformation era

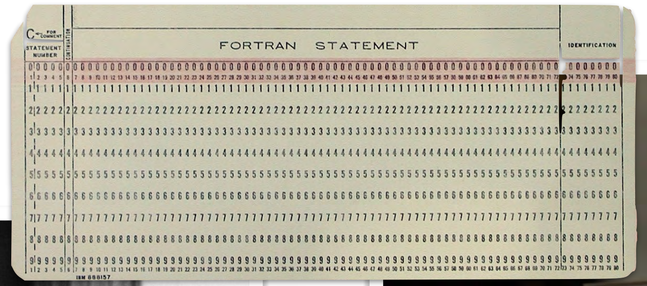
no system software

- 1940s: programming based on dials & switches
  - 1950s: single user, punched cards, paper tape

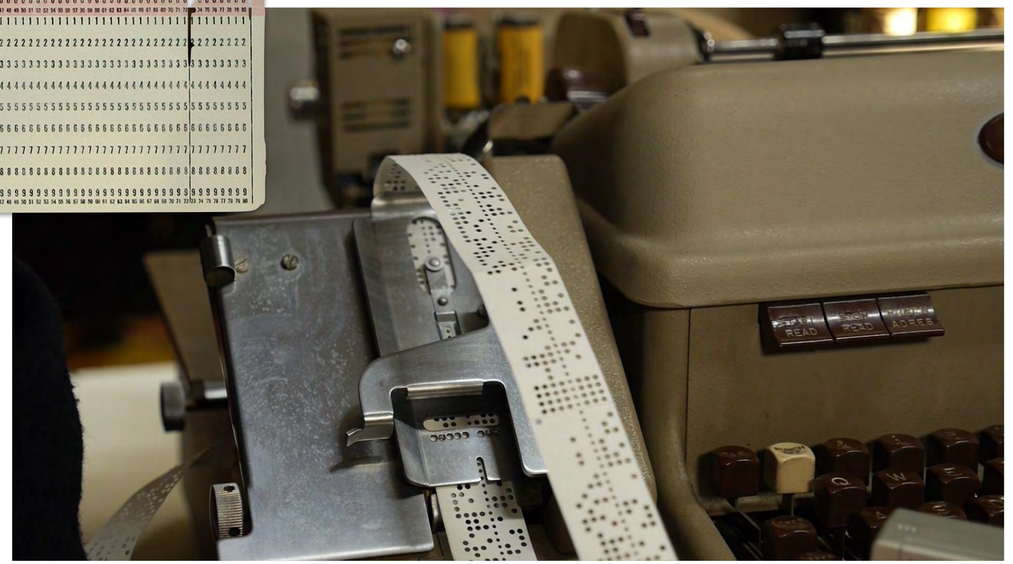
ENIAC: 30 tons, 200 kilowatts





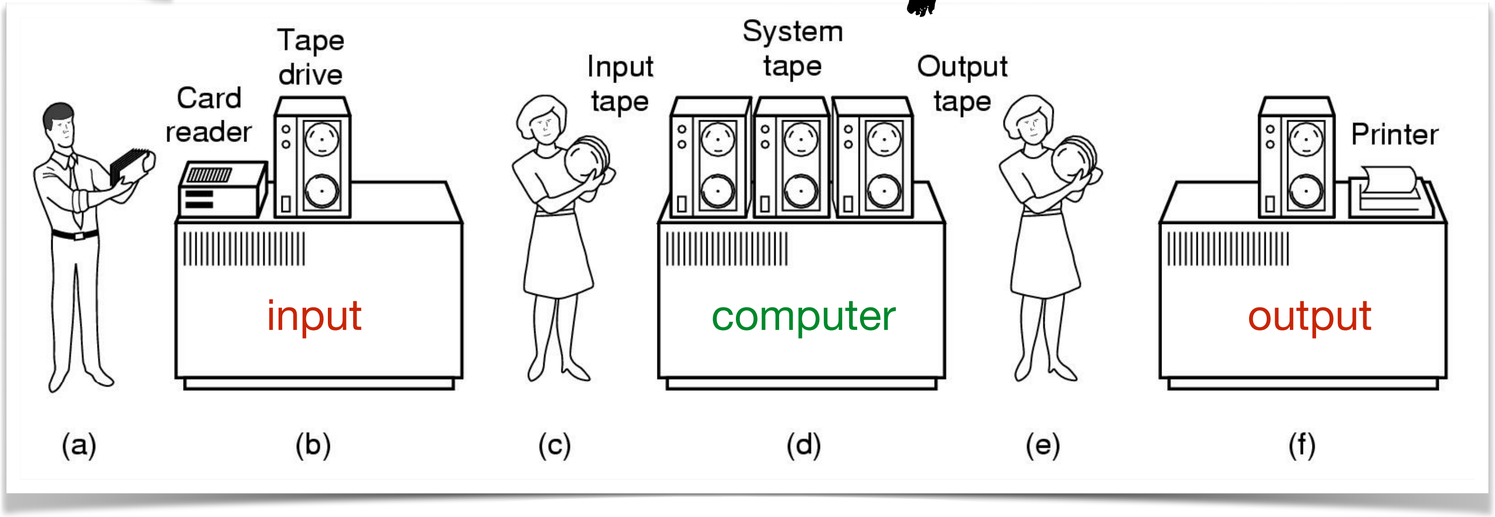






1960s

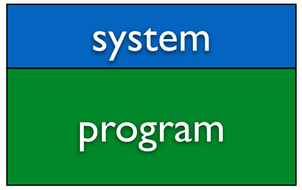
batch systems



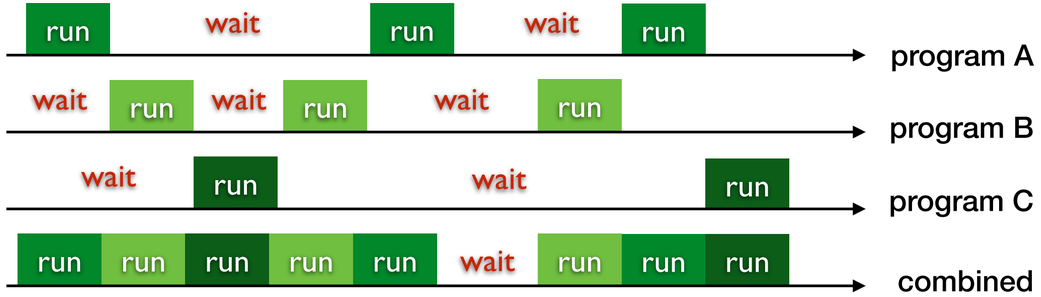
(a) put cards into reader
(b) read cards to tape
(c) put input tape on computer
(d) perform the computation
(e) put output tape on printer
(f) print output tape on paper

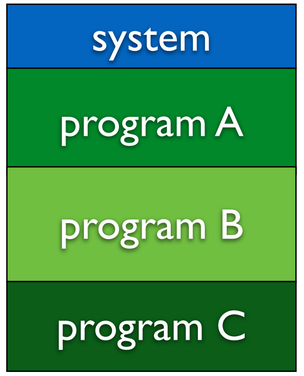
- first uni-programmed batch systems





- then multi-programmed batch systems





1970s multi-user & time-sharing

from Multics...

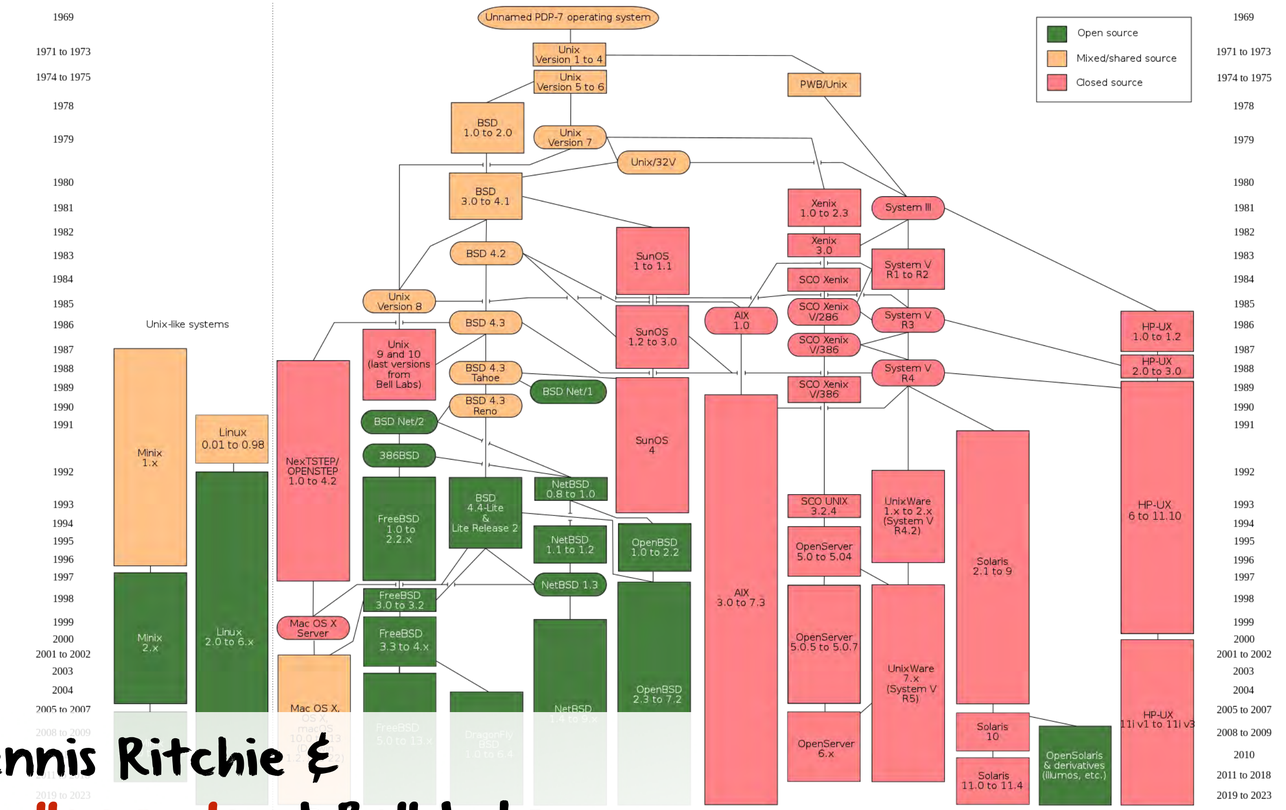




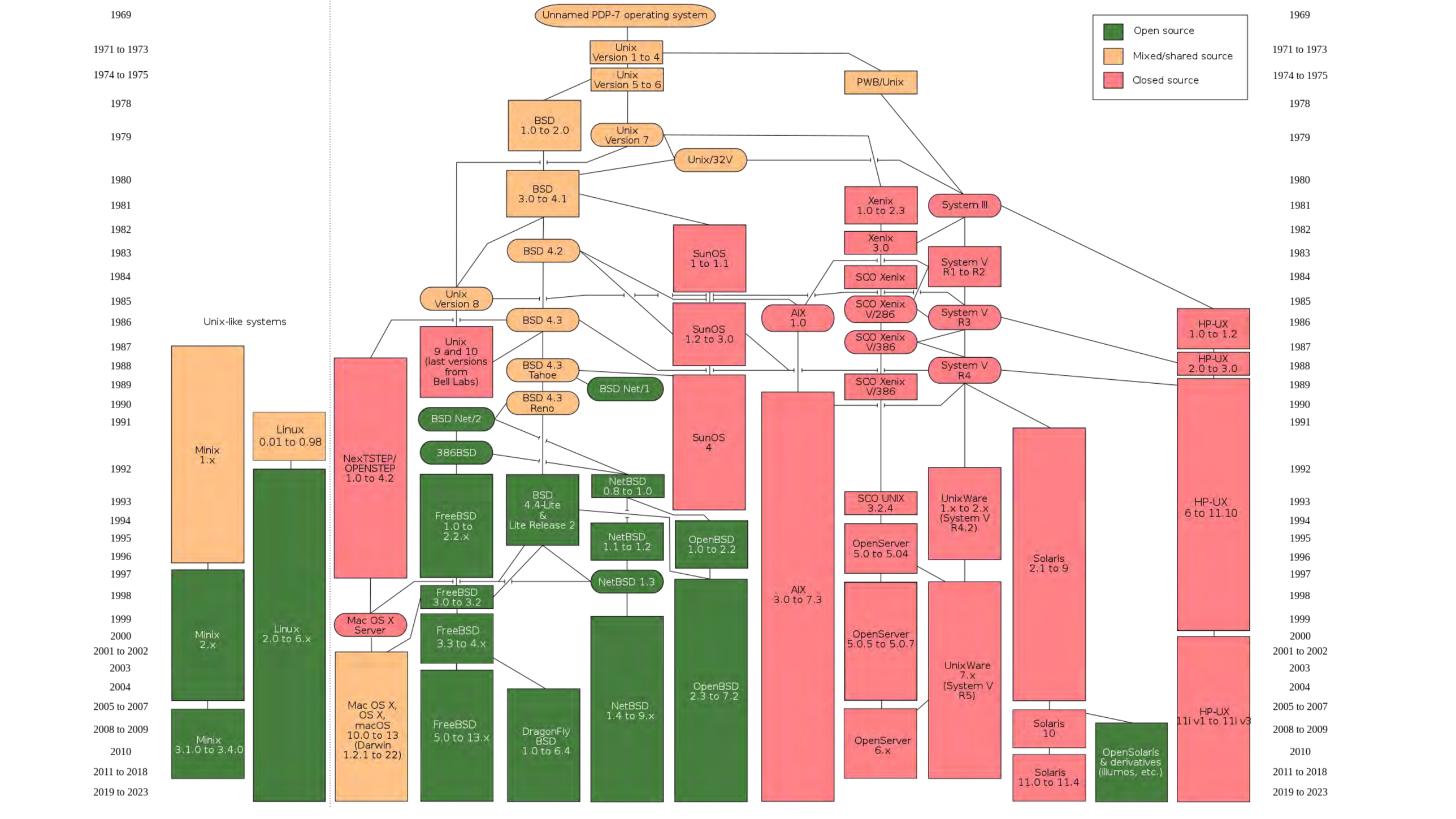
...to Unix

- 1960s: disasters... but great learning & innovations
  - OS/360: years behind schedule, shipped with 1000 known bugs
  - Multics: started in 1963, working in 1969, far too complex
- 1970s: finally mastering complexity thanks to:
  - higher level structured languages (Algol, C, Pascal, etc.)
  - portable operating systems code (C was invented for that)
  - stacking layers (kernel, compilers, libraries, etc.)





- after the Multics "disaster", Ken Thompson, Dennis Ritchie & others decided to redo the work on a much smaller scale at Bell Labs
  - in 1972, Unix was rewritten from assembly language to C programming language, resulting in the first portable operating system
    - in 1975, Ken Thompson was on sabbatical at Berkeley and worked with Bill Joy, then a graduate student, which eventually led to BSD Unix
      - in 1980, the DARPA project chose BSD Unix as basis for DARPANet
        - in 1982, Bill Joy joined Sun Microsystems six months after its creation as full co-founder and extended BSD Unix to make it a networked operating system


Open source
Mixed/shared source
Closed source
Unix-like systems
1969
1971 to 1973
1974 to 1975
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001 to 2002
2003
2004
2005 to 2007
2008 to 2009
2010
2011 to 2018
2019 to 2023
Unnamed PDP-7 operating system
Unix Version 1 to 4
Unix Version 5 to 6
PWB/Unix
BSD 1.0 to 2.0
Unix Version 7
Unix/32V
BSD 3.0 to 4.1
Xenix 1.0 to 2.3
System III
Xenix 3.0
BSD 4.2
SunOS 1 to 1.1
System V R1 to R2
SCO Xenix
Unix Version 8
SCO Xenix V/286
AIX 1.0
System V R3
BSD 4.3
SunOS 1.2 to 3.0
SCO Xenix V/386
HP-UX 1.0 to 1.2
Unix 9 and 10 (last versions from Bell Labs)
BSD 4.3 Tahoe
BSD Net/1
System V R4
HP-UX 2.0 to 3.0
BSD 4.3 Reno
BSD Net/2
SunOS 4
SCO Xenix V/386
Minix 1.x
Linux 0.01 to 0.98
NexTSTEP/ OPENSTEP 1.0 to 4.2
386BSD
AIX 3.0 to 7.3
HP-UX 6 to 11.10
NetBSD 0.8 to 1.0
SCO UNIX 3.2.4
UnixWare 1.x to 2.x (System V R4.2)
Solaris 2.1 to 9
FreeBSD 1.0 to 2.2.x
BSD 4.4-Lite & Lite Release 2
NetBSD 1.1 to 1.2
OpenBSD 1.0 to 2.2
OpenServer 5.0 to 5.04
Minix 2.x
Linux 2.0 to 6.x
NetBSD 1.3
FreeBSD 3.0 to 3.2
OpenServer 5.0.5 to 5.0.7
UnixWare 7.x (System V R5)
Mac OS X Server
FreeBSD 3.3 to 4.x
Solaris 10
HP-UX 11i v1 to 11i v3
Minix 3.1.0 to 3.4.0
Mac OS X, OS X, macOS 10.0 to 13 (Darwin 1.2.1 to 22)
FreeBSD 5.0 to 13.x
DragonFly BSD 1.0 to 6.4
NetBSD 1.4 to 9.x
OpenBSD 2.3 to 7.2
OpenServer 6.x
OpenSolaris & derivatives (illumos, etc.)
Solaris 11.0 to 11.4

how did we get there?

the invention of the microprocessor



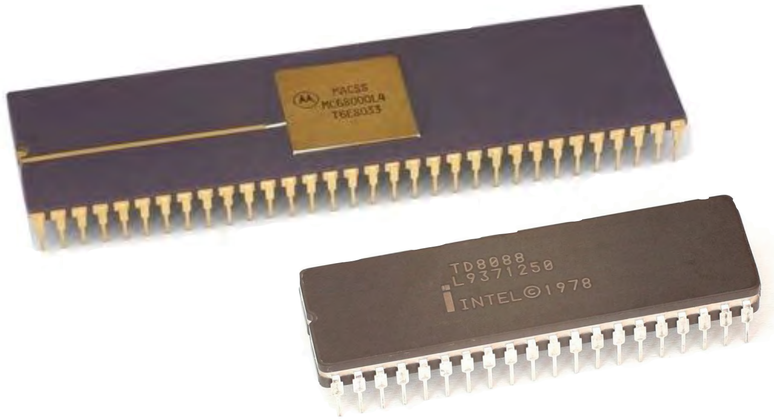
IBM System/360

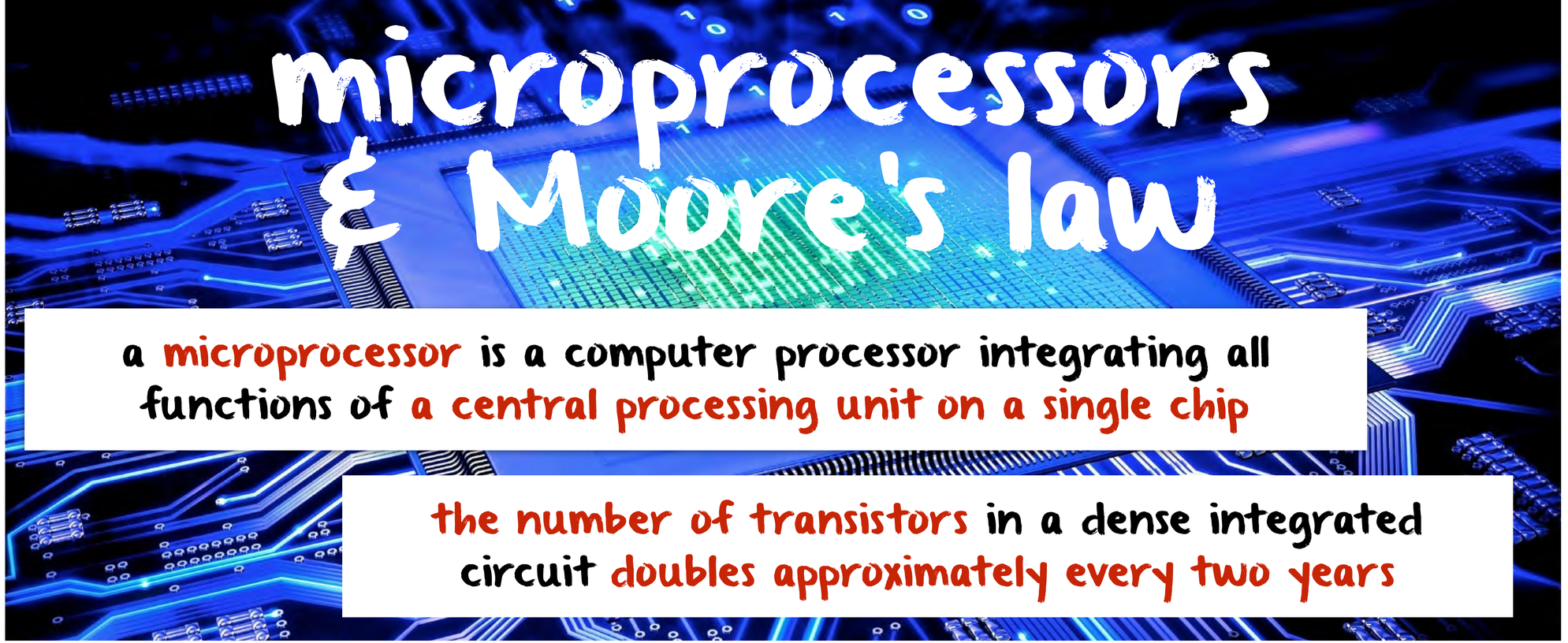


DEC PDP-11



DEC PDP-11 Processor





microprocessors & Moore's law

a microprocessor is a computer processor integrating all functions of a central processing unit on a single chip

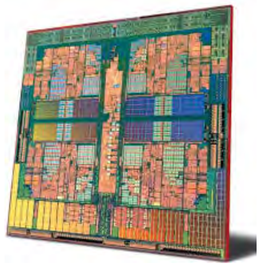
the number of transistors in a dense integrated circuit doubles approximately every two years

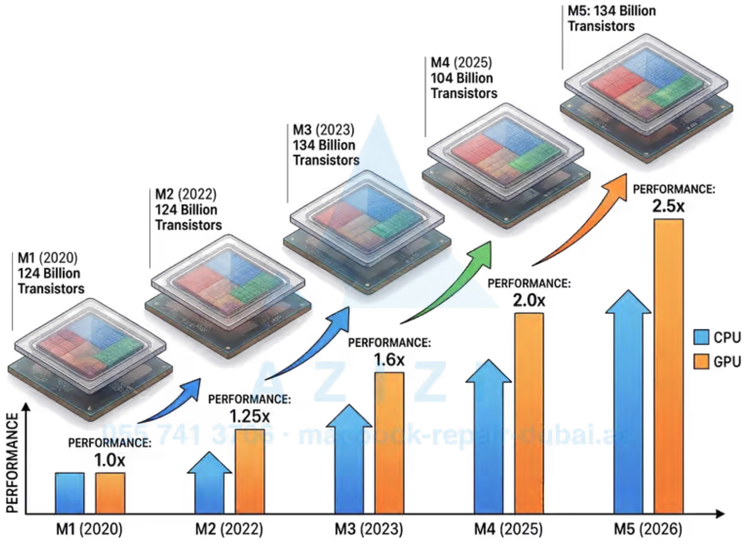
- this is unique across all engineering fields
  - transportation increased speed from 20 km/h (horse) to 2'000 km/h (concorde) in 200 years but the computer industry has been doing this every decade for the past 60 years
    - the advent of the microprocessor triggered the decline of mainframes and led to the personal computer revolution

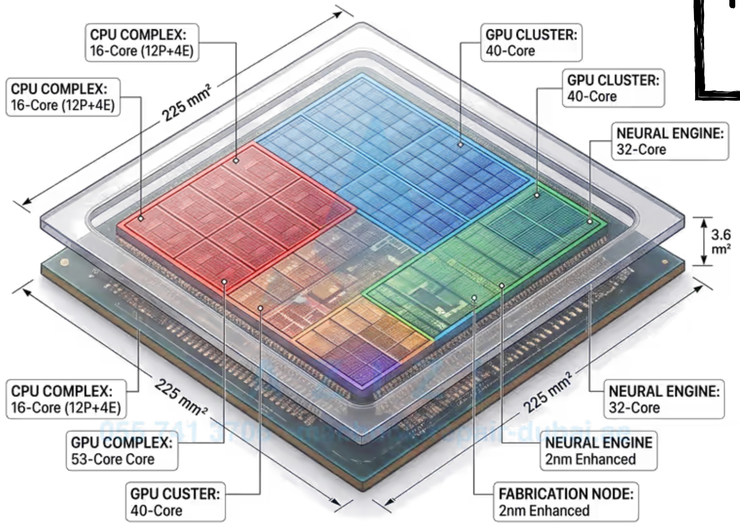
writing system software is about mastering exponential complexity

As long as there were no machines, programming was no problem at all; when we had a few weak computers, programming became a mild problem and now that we have gigantic computers, programming has become an equally gigantic problem. In this sense the electronic industry has not solved a single problem, it has only created them - it has created the problem of using its products.

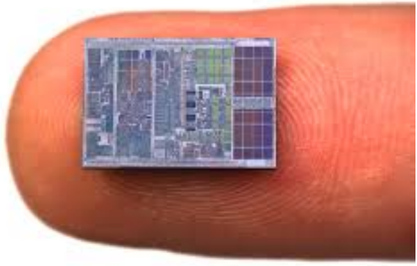
Edsger Dijkstra, The Humble Programmer. Communications of the ACM, vol. 15, no. 10. October 1972. Turing Award Lecture.







the industry is now going multicore



acceleration













1980 1990 2000 2010

1980s: one man, one computer

- workstation, personal computers
- graphical user interfaces

1990s: the network is the computer

- the internet accessible to all
- distributed operating systems

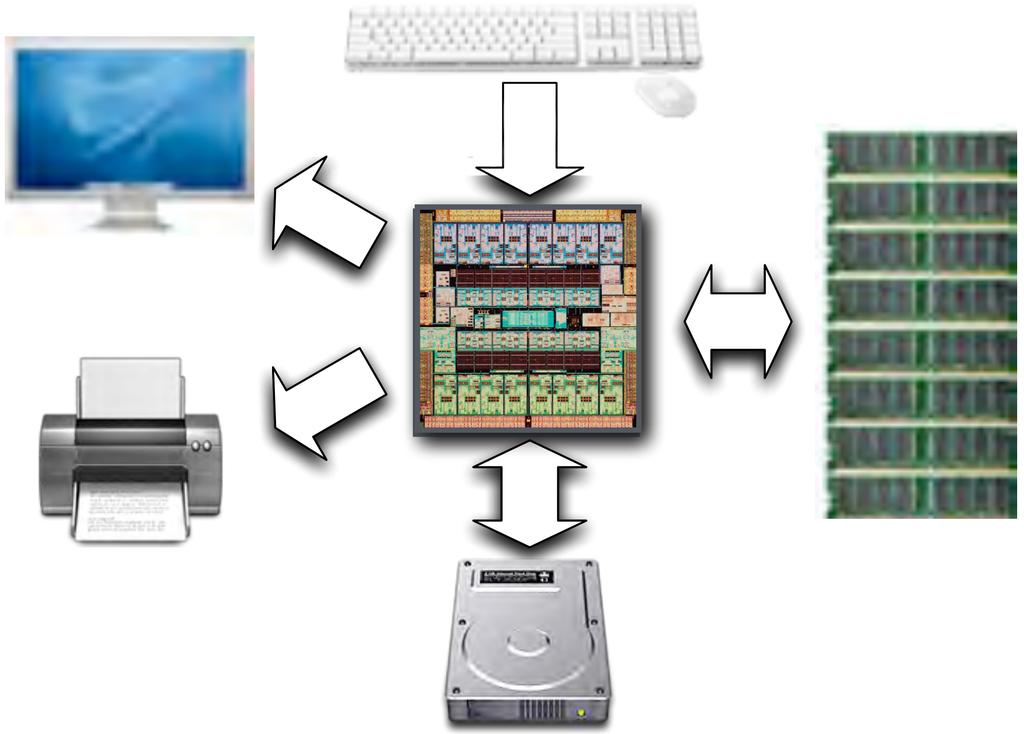
2000s: my phone is my computer

- smartphones & tablets as computers
- generalization of wireless networks

2010s: everything is a computer

- smart objects & the internet of things
- personal networks connected to the cloud

operating system

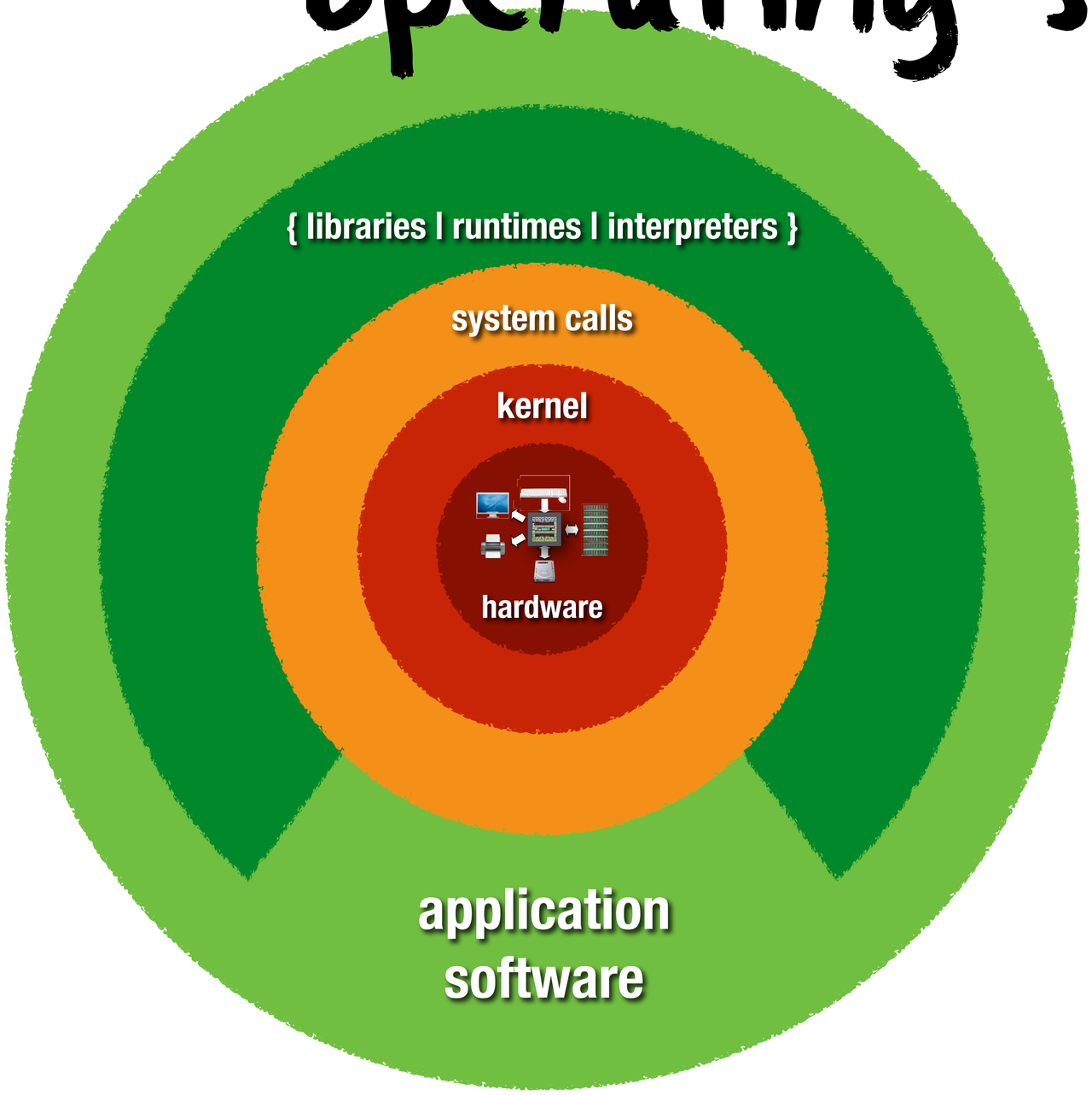


controls the access to hardware resources

(cpu, memory, input/output devices, etc.) and acts as

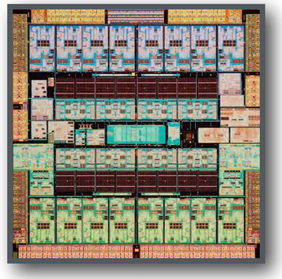
an interface with application software

operating system

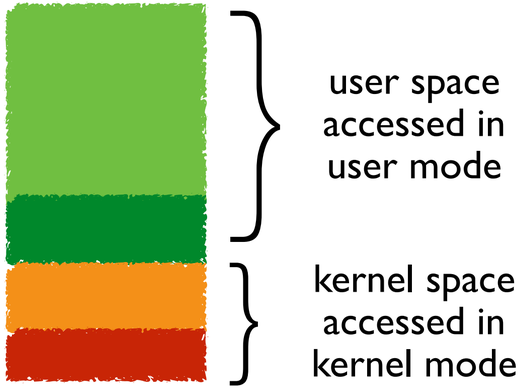


processor modes

- kernel mode (system)
- user mode (application)



memory protection



operating system

resource management & virtualisation

- **cpu**: process management
  - **memory**: memory management
    - **input/output**: i/o management
      - keyboard, mouse, display
      - touch screen, haptic interface, network
      - printer, audio device, connectors (usb, dvi, etc.)
      - compass, accelerometer, global positioning system
      - etc...
    - **storage**: storage and file management

	reality (physical resources)	abstraction (virtual resources)
CPU	***n parallel cores***	***m concurrent threads***, *with* $m \gg n$
memory	***subset of 2^k addressable memory*** *on a* ***k*** *bits machine, e.g., for* $k = 64$, *this is typically* ***8 to 32 gigabytes***	***full 2^k addressable memory*** *for* $k = 64$, *this is* ***16 exabytes*** $\cong 16 \times 10^6$ ***terabytes*** $\cong 16 \times 10^9$ ***gigabytes***
	in addition, each thread can access the full ***2^k addressable memory*** *as if it was for its exclusive use*	
storage	*hard disk drive (hdd), solid state drive (ssd), usb keys, etc...*	*file system offering persistency*
network	***i network interfaces***, *e.g., wifi, ethernet*	***j network connections***, *with* $j \gg i$

filesystems & command shells

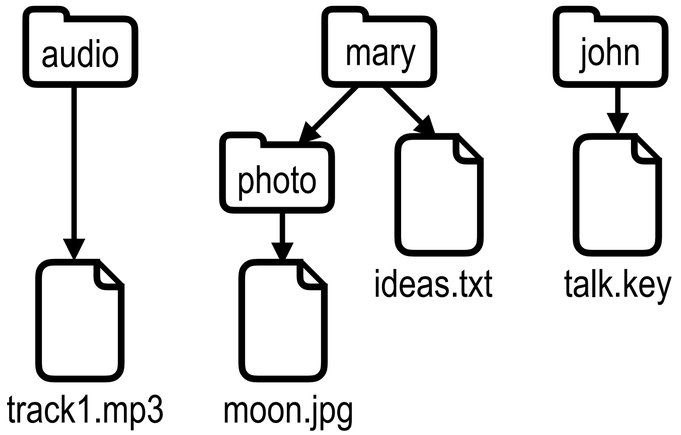
a filesystem is a part of the operating system that allows users to manipulate data stored on some persistent storage, typically a disk

a terminal is a program that allows users to interact with the operating system using a command line interface known as a shell

a shell is an interpreter for a specific scripting language that can be used either interactively, via a terminal, or launched as a program by providing it with a script file containing commands

a scripting language is a programming language

filesystems



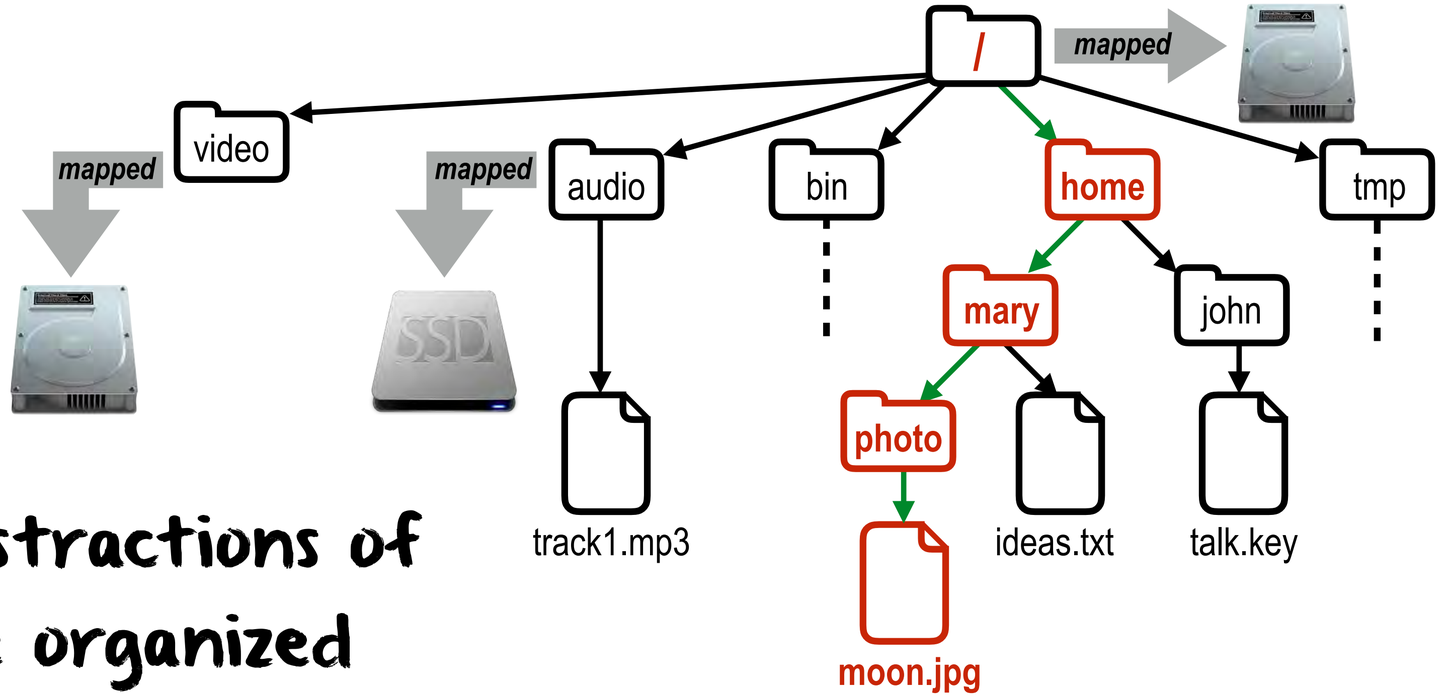
a filesystem is based on the abstractions of files and directories, which are organized and accessed via paths in a namespace

a file is an abstraction representing a sequence of bits stored on some persistent storage; this sequence of bits is the content of the file

a directory is an abstraction representing a group of files and directories; the references to those files and directories constitute the content of the directory

in addition to their content, files and directories contains attributes, among which their name, their type, their size, their access rights, etc.

filesystems



a filesystem is based on the abstractions of files and directories, which are organized and accessed via paths in a namespace

a namespace maps files and directories to their physical location (disk) on the persistent storage, via a hierarchy of names organized as a tree

a path is a sequence of names (separated by some special character) that specifies a unique location in the filesystem, starting from the root of the namespace tree

command shells

```
TestJava — -bash — ttys001
wallace-palace:~ garbi$ ls
Applications            Development             Icon?                   Movies                  Resources               Technology
Arcade                  Documents               LaTeX                   Music                   Scanning                Traveling
Archives                Downloads               Library                 PhD Students            Science                 Videos
Arts                    Dropbox                 Luniistore              Pictures                Sites
Creative Cloud Files    Entrepreneurship        Manuals                 Products                Stacks
Culture                 Events                  Matchmore               Public                  SwitchDrive
Desktop                 FCE                     Misc                    Research                Teaching
wallace-palace:~ garbi$ cd Development/
wallace-palace:Development garbi$ ls
butter                  flutter                 flutter_apps            react-native            technical-ios-app
wallace-palace:Development garbi$ ls
butter                  flutter                 flutter_apps            react-native            technical-ios-app
wallace-palace:Development garbi$ mkdir TestJava
wallace-palace:Development garbi$ ls
TestJava                butter                  flutter                 flutter_apps            react-native            technical-ios-app
wallace-palace:Development garbi$ cd TestJava/
wallace-palace:TestJava garbi$ ls
wallace-palace:TestJava garbi$ pwd
/Users/garbi/Development/TestJava
wallace-palace:TestJava garbi$ ls
wallace-palace:TestJava garbi$
```

typical terminal of the unix operating system

Basic Bash Commands

ls	list files and directories in the current working directory
ls -la	list all files and directories with details in the current working directory
cd *directory*	change the working directory to be *directory*
cd ~	change the working directory to be your home directory
cd ..	change the working directory to be the parent directory
pwd	print the current working directory
mkdir *directory*	create a new directory named *directory*
more *textfile*	display the content of the file named *textfile* one page at a time
man *command*	display help about the command named *command*

the terminal

runs

the command shell

that interprets

your command lines

you will often launch your programs from the command line

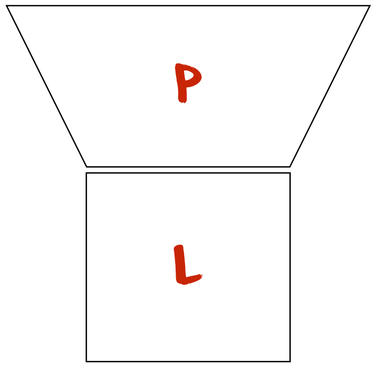
executions and interpreters

concept

program P

written in

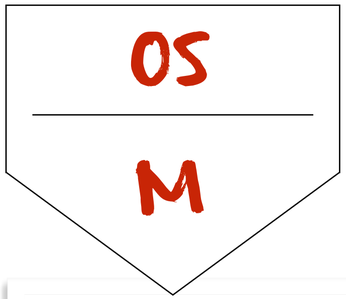
language L



operating system OS

controlling machine

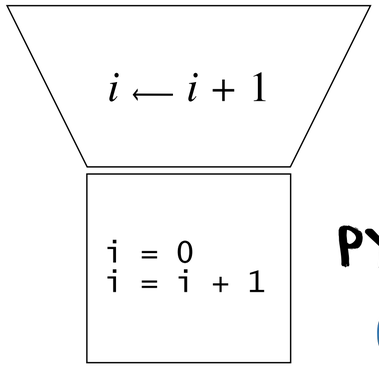
executing language M



examples

an addition

written in

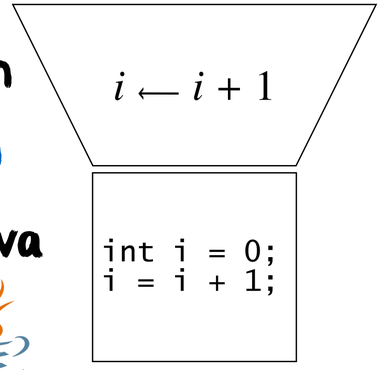


python



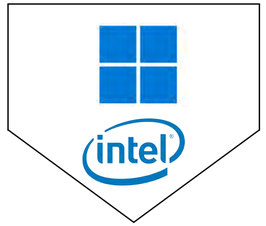
java







Samsung Galaxy
running Android
on ARM



HP Laptop
running Windows
on Intel



Oracle Server
running Solaris
on SPARC



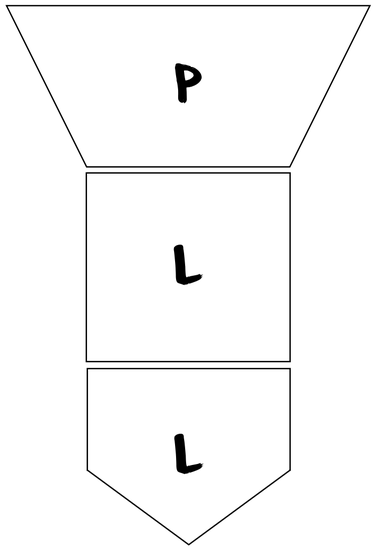
MacBook Pro
running macOS
on M5 (ARM)

machine language M ⇔ instruction set ⇔ byte code

executions and interpreters

concept

program P
written in
language L
running on
machine L

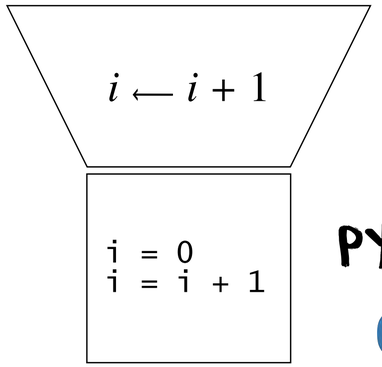


program language must match machine language

we forget about the operating system for now

examples

an addition
written in

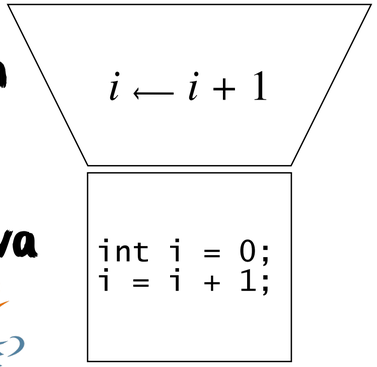


python



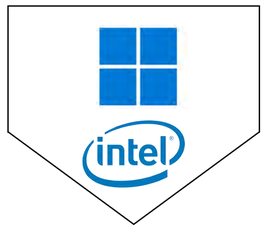
java







Samsung Galaxy
running Android
on ARM



HP Laptop
running Windows
on Intel



Oracle Server
running Solaris
on SPARC

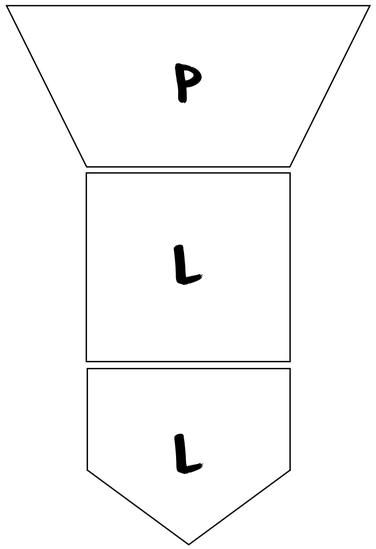


MacBook Pro
running macOS
on M5 (ARM)

executions and interpreters

concept

program P
written in
language L
running on
machine L



program language must match machine language

we forget about the operating system for now

examples

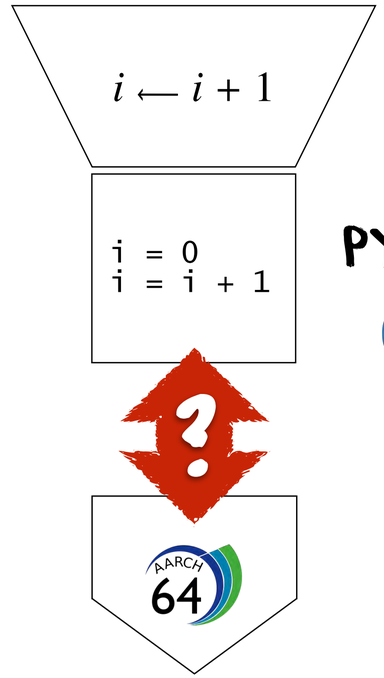
an addition
written in

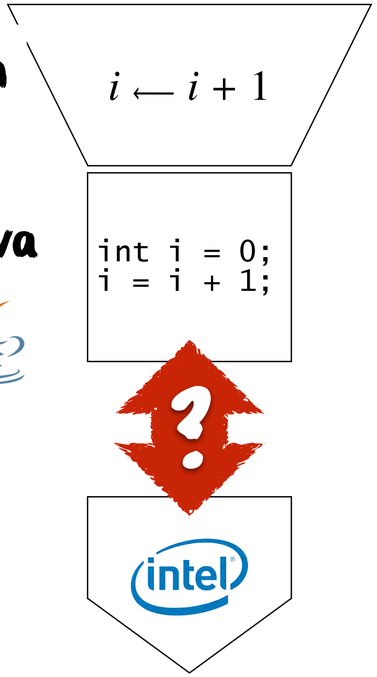
python



java



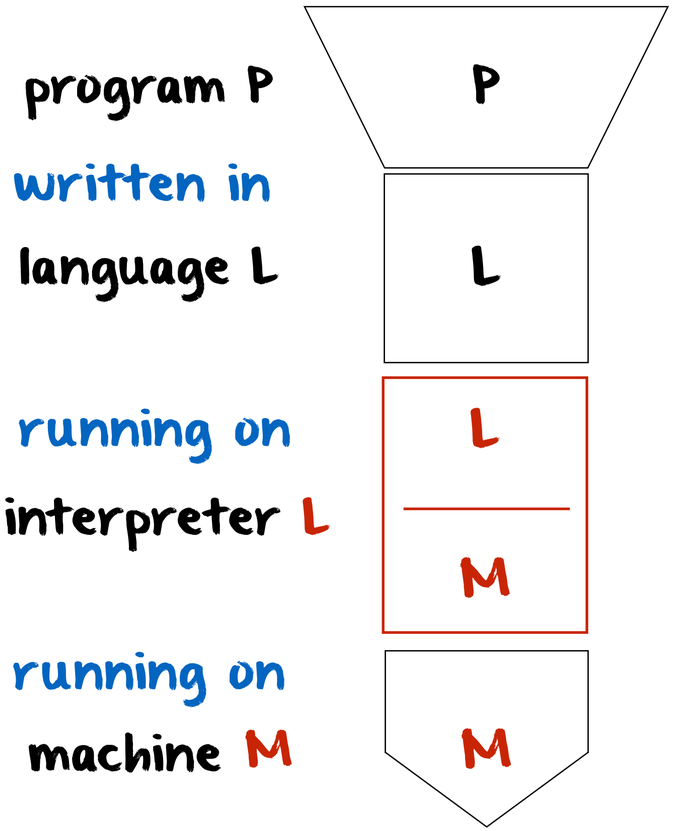




problem!

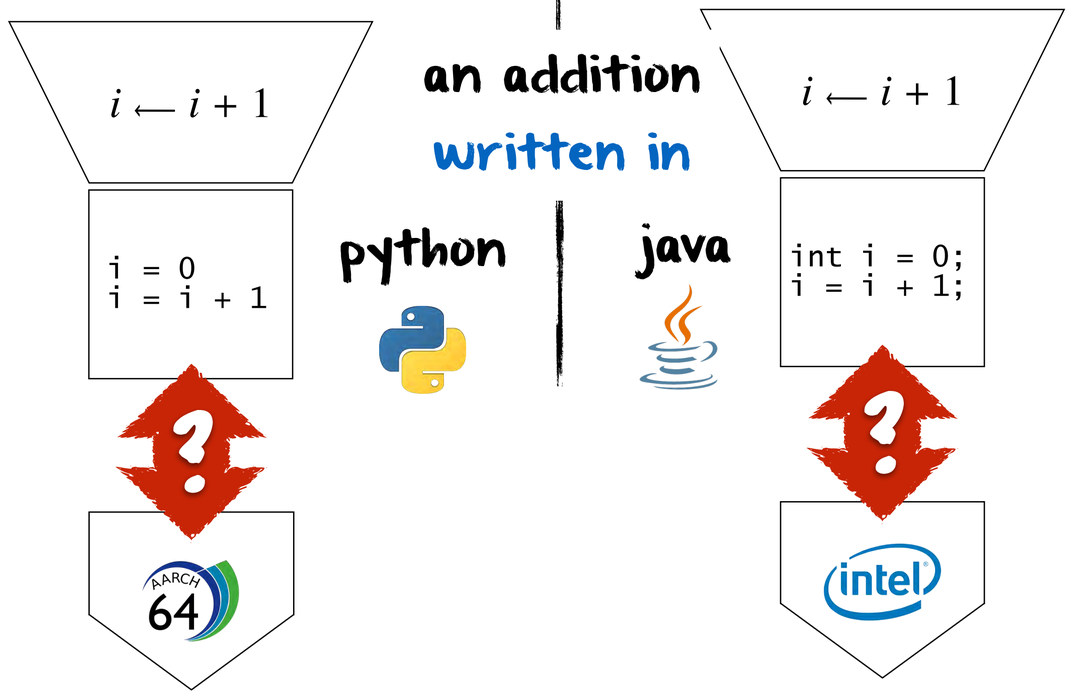
executions and interpreters

concept



an interpreter dynamically translates language L into language M

examples



solution!

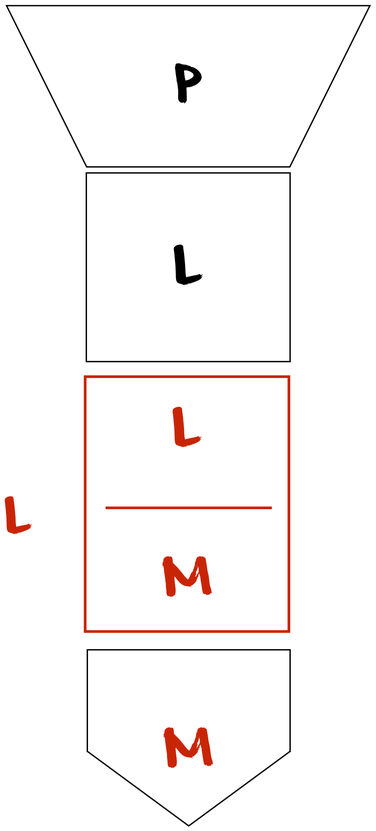
executions and interpreters

concept

program P

written in

language L



running on

interpreter L

running on

machine M

an interpreter dynamically translates language L into language M

examples

$i \leftarrow i + 1$

```
i = 0
i = i + 1
```



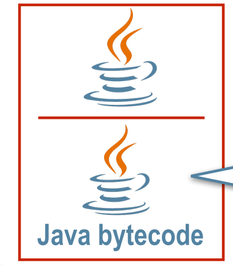


an addition written in

python | java

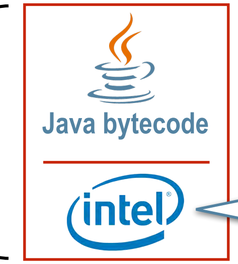
$i \leftarrow i + 1$

```
int i = 0;
i = i + 1;
```

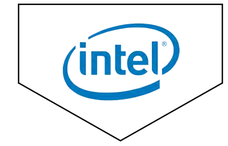



```
bipush
istore_1
getstatic #2
iload_1
invokevirtual #3
return
```

java virtual machine




```
MOV EAX, 42
PUSH EAX
CALL _function
ADD ESP, 4
CMP EAX, 0
JNE _label
XOR EDX, EDX
RET
```



interpreter ⇔ emulator ⇔ virtual machine



what's a compiler

a program that translates
human-understandable source code to
machine-understandable byte code



0010010100101011000100101011001101001110011111001101010...

```
LDR   R3   R1
ADD   R3   R1   R2
STR   R3   R1
```

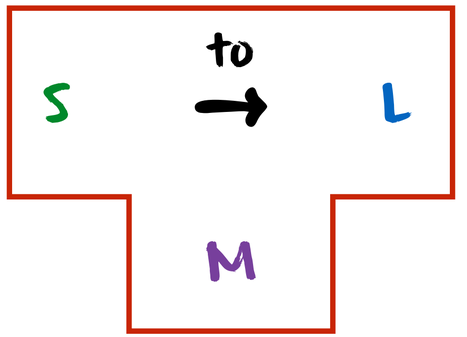
Java bytecode

what's a compiler

concept

compiler translating

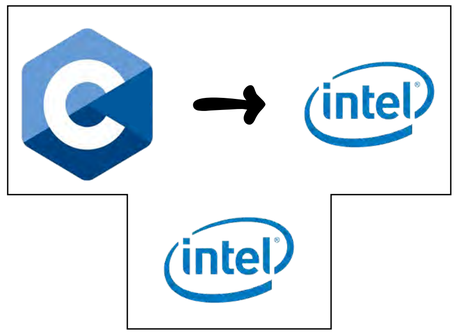
source language

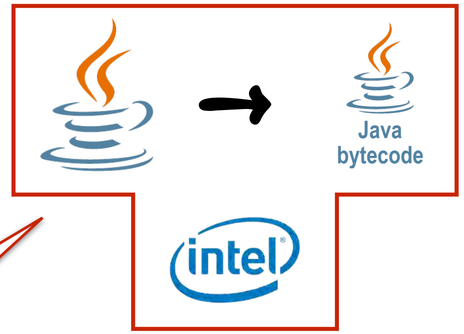


target language

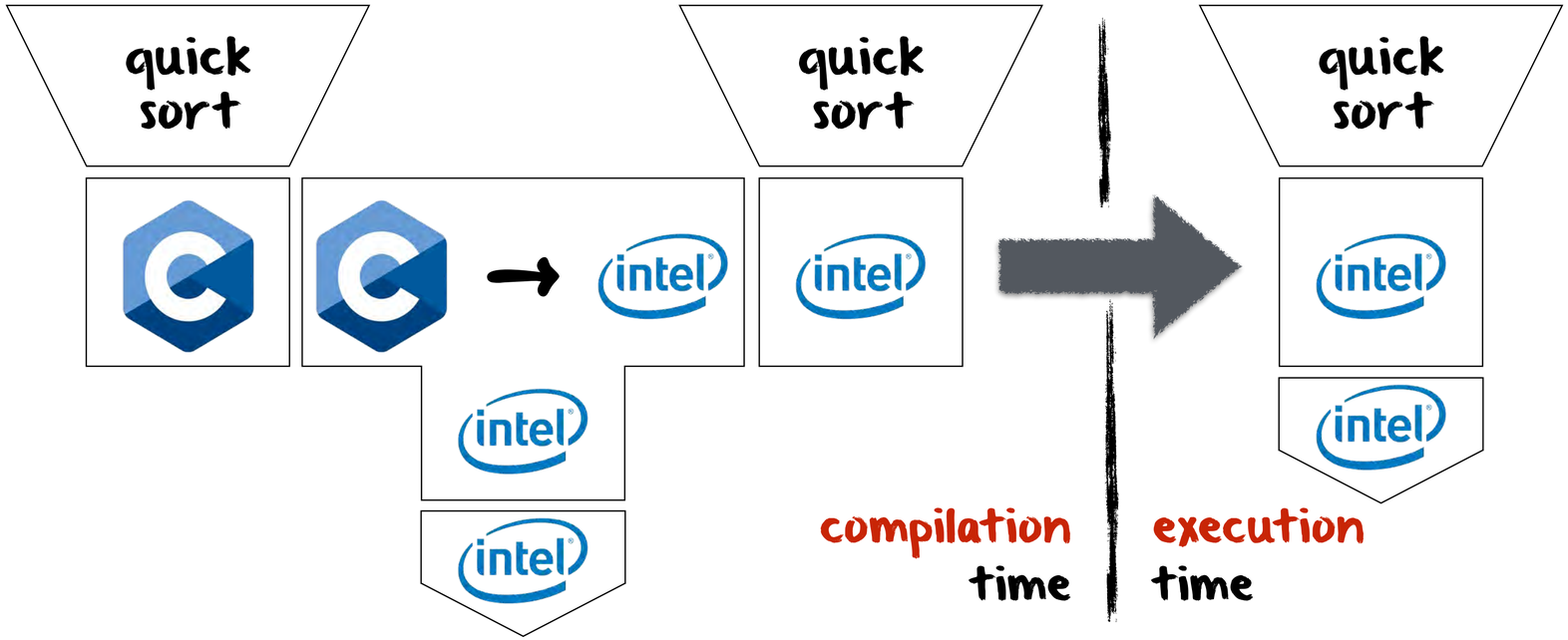
implemented in M

examples



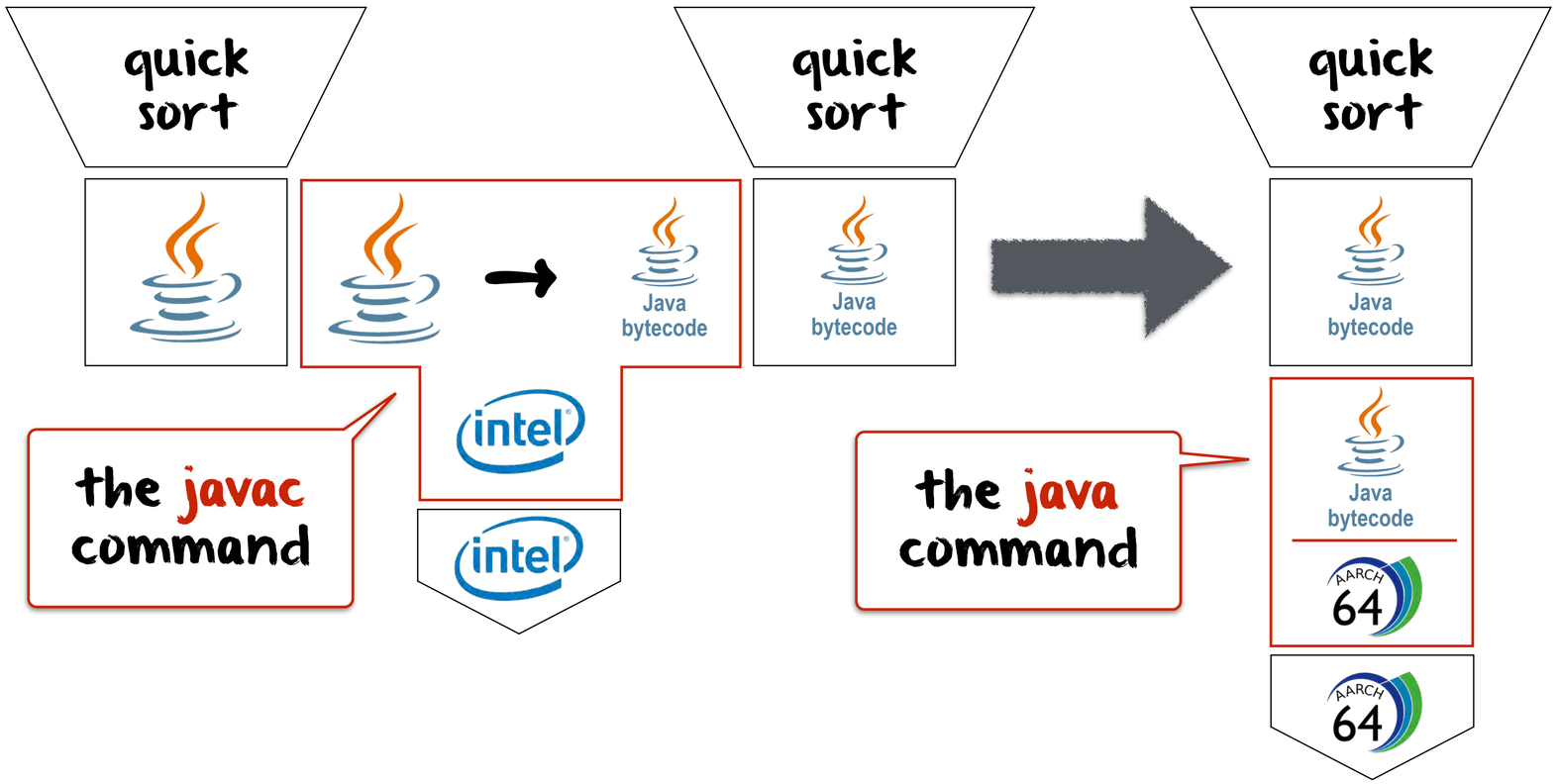


this is a cross-compiler

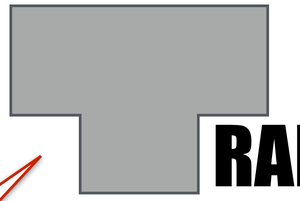


what's a compiler

the example of java



static vs. dynamic



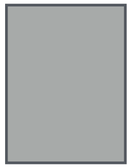
RANSLATION

the javac command

the translation occurs at compile time, before the execution, while the program is static

---

the java command



NTERPRETATION

the interpretation occurs at run time, during the execution, while the program is dynamic



what are runtime systems & libraries?



a library contains predefined bricks (functions, objects, etc.) that help create software, e.g., strings, dates, lists, input/output functions, etc.



a runtime system is the mortar that glues the various parts of software during execution

---

```
public class HelloWorld {

   public static void main(String[] args) {
      System.out.println("Hello World!");
   }

}
```

where does System.out come from?

where does String come from ?

where is args stored?

how is “Hello, world!” passed to println(…)?



what are runtime systems & libraries?

```
public class HelloWorld {

    public static void main(String[] args) {
        System.out.println("Hello World!");
    }

}
```

where does String come from ?

where does System.out come from?

where is args stored?

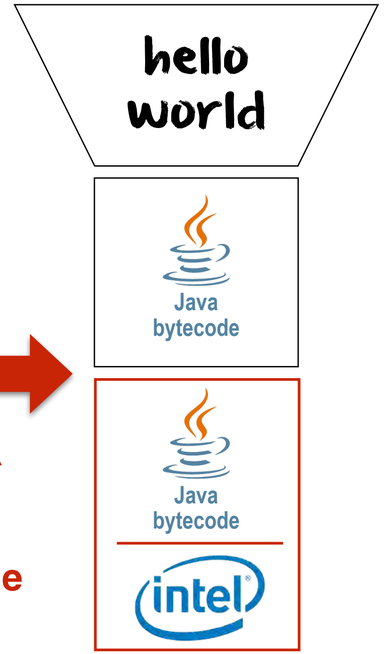
how is “Hello, world!” passed to println(…)?

---

the answer is...



…the java **runtime system & library**



putting it all to work on the command line

HelloWorld

HelloWorld.java
HelloWorld.py

```
my-computer ~ % cd HelloWorld
my-computer HelloWorld % ls -l
total 16
-rw-r--r--  1 garbi  staff  114 Sep 10 13:40 HelloWorld.java
-rw-r--r--  1 garbi  staff   21 Sep 10 13:41 HelloWorld.py
```



java

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello World!");
    }
}
```

```
my-computer HelloWorld % javac HelloWorld.java
```

compile
java source
to java byte code

```
my-computer HelloWorld % ls -l
total 24
-rw-r--r--  1 garbi  staff  425 Sep 10 14:19 HelloWorld.class
-rw-r--r--  1 garbi  staff  114 Sep 10 13:40 HelloWorld.java
-rw-r--r--  1 garbi  staff   21 Sep 10 13:41 HelloWorld.py
```

```
garbi@wallace-palace HelloWorld % java HelloWorld
```

interpret
java byte code

```
Hello World!
```

python



```
if __name__ == "__main__":
    print("Hello World!")
```

```
garbi@wallace-palace HelloWorld % python HelloWorld.py
```

directly
interpret
python source

```
Hello World!
```